

LU 3: Linear Data Structures and Algorithms

1. Array

DSA & C++

<https://shorturl.at/psDW7>

3.1.0 Array

Suppose we want to store the marks of 20 students. To get this goal, you have two options.

1) Construct 20 variables and use them to assign individual score for each of the 20 students.

```
int marks1, marks2, marks3, marks4, ..... Marks20;  
/*Then initialise each variable */  
marks1=16, marks2=20, marks3=15, marks4=14,.....marks20=10;
```

*If you have 120 students, then you need 120 different variables
marks1=16, marks2=20, marks3=15, marks4=14..... Marks120;
/* and you initialise them as follow*/
marks1=16, marks2=20, marks3=15, marks4=14,.....marks120=10;*

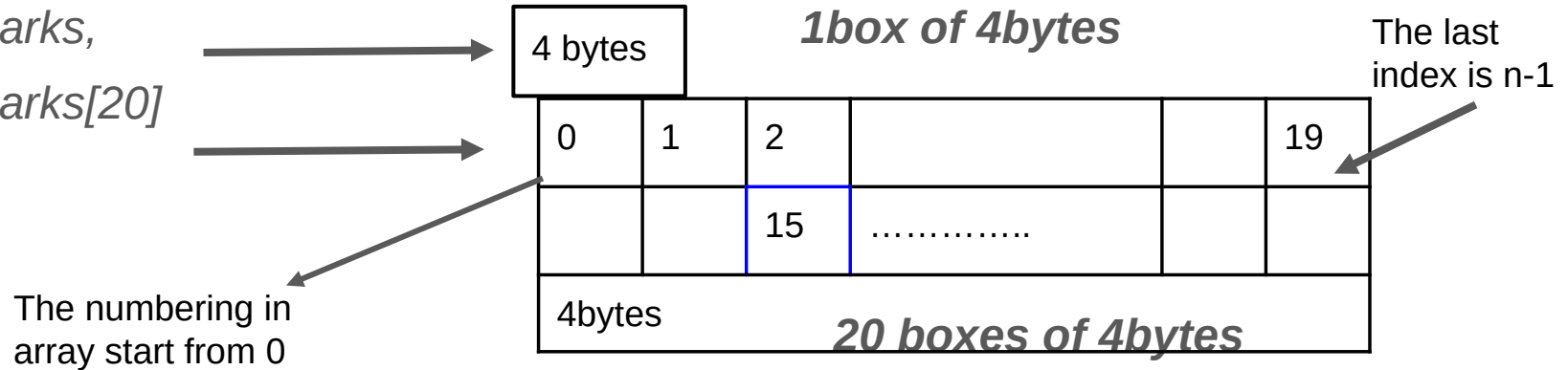
Second option

2) Create an array of 20 items and use them. To access the different marks we have to only change the index. Of course, the second approach is more convenient than the first approach.

Memory allocation

`int marks,`

`int marks[20]`



To initialise an element of the array , `marks[2]=15` => As the third element in the box. The value of each array element in this array **MUST** be an integer value. The number of values between braces `{ }` shall not be greater than the number of elements in the array.

1D Array and Algorithms

3.1.1 What is an Array? What is one dimensional array?

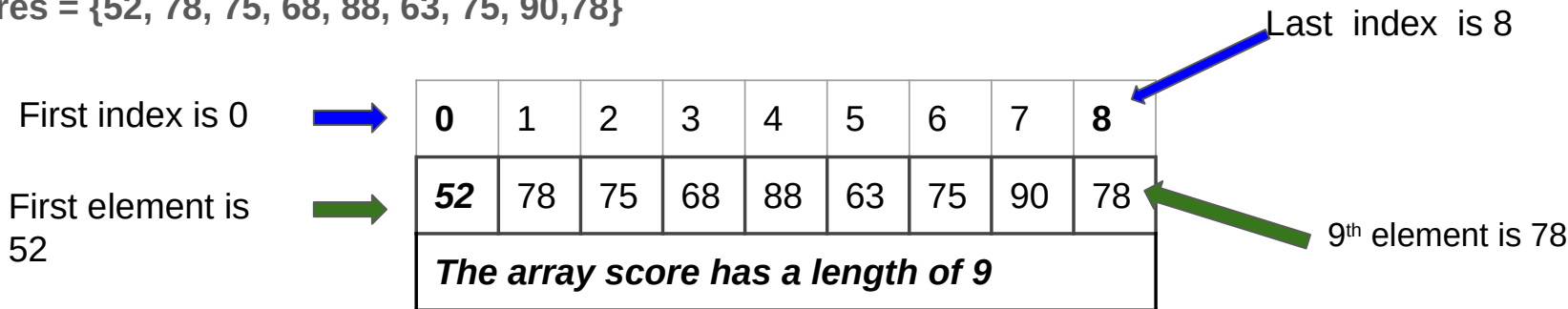
The **array** is a collection of elements of the same type.

A **one dimensional array** is a group of contiguous memory locations identified by the same name for storing data the same type.

Array items are typically stored in a sequence of computer memory locations, but to discuss them, we need a convenient way to write them down on paper. We can just write the items in order, separated by commas and enclosed by square brackets. Array helps the programmer in minimizing the variables. For Instance

{52, 78, 75, 68, 88, 63, 75, 90, 78}

is an example of an array of integers value for students' score. If we call this array **scores**, we can write it as: **scores = {52, 78, 75, 68, 88, 63, 75, 90, 78}**



This **array score** has 9 items, and hence we say that its **size is 9**. However, we work with arrays in programming, we more often start from **0**. Thus, for our array score, its positions are 0, 1, 2, 3, . . . , 7, 8. The score element in the 8th position is 78, and we use the notation `score[8]` to denote this element.

3.1.2 Declaration of Arrays

An array occupy space in memory. Therefore, declaring an array is the same as declaring other variables only that a computer reserves contiguous memory locations enough to store the number of elements.

The general syntax of declaring an array is:

Arrayname: Array [elements] of datatype

e.g.

Scores: Array[10] of integer;

4 factors on declaration one dimensional array

Regardless of language used to implement arrays, the following are factors that need to be considered.

Array name: Decide on a suitable array name that indicates several elements are to be stored e.g. scores.

Data type of elements: An array can only hold elements of the same data type.

Size of array: The size of an array determines the maximum number of values that an array will hold.

Dimension: An array can be one-dimensional list or multidimensional such as a table (matrix).



3.1.3 Reading/Storing array elements

To access a particular element in an array, we specify the name of the array and the position number (**index**) of the element which is also called a **subscript**. The index is a position number that must be an integer or an integer expression.

To store (read) a value into an array, you need to know the name of the array and the index of the element. Then a **READ** function may be applied to the element.

For example **READ Scores [4]** stores a value in the **fifth** location of the score array. Multiple values may be read into several elements using a FOR loop.

NOTE

The scores array is set to store 10 elements of integer type.

The for loop uses index as a counter to continuously store ten elements 0 to 9.

The for loop is eliminated once the ten elements have been read into the array.

```
BEGIN
```

```
    SET scores=Array[10]of Integer
```

```
    FOR index=0 TO 9 DO
```

```
        READ
```

```
        scores[index];
```

```
        index = index +
```

```
        1
```

```
    LOOP
```

```
END FOR
```

```
END
```



3.1.4 Writing/Printing Array Elements

To write (display) elements from an array, use a write function together with the arrayname and location (index) of the element.

To display a single value of an array you must provide the array name and index to the write operation.

For example, the value in Scores [1] may be displayed by using PRINT Scores[1] .

To display multiple values such as the 10 elements in the Scores array, use the FOR loop by setting the initial value to 0 and the upper limit to 9

```
BEGIN
```

```
    SET scores[9]={52, 78, 75, 68, 88, 63, 75,  
90,78}
```

```
    SET len=9
```

```
    FOR index=0 TO len-1 DO
```

```
        PRINT score[index]
```

```
        index=index+1
```

```
    END FOR
```

```
END
```



3.1.5 Insertion an element in array algorithm

Insert operation is to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array.

Here, we see a practical implementation of insertion operation, where we add data at the end of the array

```

1. BEGIN
2.     SET scores[]={52, 78, 75, 68, 88, 63, 75, 90,78}
3.     SET item=10, k=3, len=9, j=len-1
4.     Repeat steps 5 and 6 while j>=k
5.     SET scores[j+1]=scores[j]
6.     SET J = J -1
7.     SET scores[k]=item
8.     FOR index=0 TO len+1 DO // FOR i ← 1 TO len+1
DO
9.         PRINT scores[index]
10.        index=index+1
11.    END FOR
  
```

3.1.6 Deletion of an element-Algorithm

Deletion refers to removing an existing element from the array and re-organizing all elements of an array.

Consider **scores** is a linear array with N elements and K is a positive integer such that $K < N$. Following is the algorithm to delete an element available at the Kth position of **scores**.

1. BEGIN
2. SET scores[9]={52, 78, 75, 68, 88, 63, 75, 90,78}
3. SET K=3, N=9, J = K
4. Repeat steps 5 and 6 while $J < N-1$
5. SET scores[J] = scores[J + 1]
6. SET J = J+1
7. SET N = N-1
8. END

3.1.7 Searching algorithms

Consider **scores** is a linear array with N elements and K is a positive integer such that $K \leq N$. Following is the algorithm to find an element with a value of ITEM using sequential search.

1. BEGIN
2. Int K
3. SET scores[9]={52, 78, 75, 68, 88, 63, 75, 90,78}
4. SET ITEM=75, N=9, J = 0,
5. Repeat steps 6 and 7 while $J < N$
6. IF scores[J] is equal ITEM THEN GOTO STEP 8
7. SET $J = J + 1$
8. SET $K=J$
9. PRINT K, ITEM
10. END

The Time complexity of Inserting, deleting and searching an element in the array is linear $O(n)$



3.1.8 C++ implementation of one dimensional array

Declaration

Declaring an array is similar to declaration of simple data types only that square [] are used to instruct the computer to reserve enough memory locations to store array elements. The general syntax of declaring a one-dimensional array is:

type arrayName[number of elements];

Where type refers to data type to be stored in the array, followed by the arrayName and number of elements.

For example, the follow array named scores stores elements of integer type

int score[10];

Initialisation

Array initialization refers to assigning elements to default values at compile time. In C++, elements of an array can be initialised during array declaration by assigning the array to list of comma-separated values enclosed in {}braces. For example,

```
int scores[10]={15,10,19,14,20,20,16,18,15,17};
```

If there are fewer initialisers than the number of elements, the remaining elements are automatically initialised to zero. For example, the elements of the house array could have been initialized to zero as follows:

```
int scores[10] = {15, 17};
```

The statement initialises the first two element to 15 and 17, and the remaining eight elements are initialised to two values followed by zeros as follows:

```
scores[10] = {15,17,0,0,0,0,0,0,0,0};
```

Initialisation(next)

It is important to note that, declaration of an array does not automatically initialise elements to zero. To automatically initialise all elements to zero, use empty braces or initialise at least the first element to zero as follows:

```
int scores [9]={0};
```

If the array size is omitted in the square bracket but elements initialised using comma-separated list of initialisers, the compiler assigns the array size enough to hold the number of elements in the list. For example, the definition below creates a five-element array:

```
int scores[ ]={16,12,13,16,15};
```

With C++, equal sign can be removed and initialisation is still valid

```
int scores[ ] {52, 78, 75, 68, 88, 63, 75, 90,78};
```

0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0

The initializer can even have no values, just the braces:

```
int scores[9]={};
```

The array score has a length of 9 and every element is zero by default.

Access: Reading values into Array Elements

To read values into a specific element of an array , use the following syntax:

```
cin>>name[n-1];
```

The statement uses the cin object to accept user input and stores the value into the element specified by n-1 offset.

```
int score[9] = {};  
//use for loop to read values into score array  
for (int i = 0; i<9; i++){  
cout<<"Please enter score No:"<<i+1<<endl;  
cin>>scores[i];  
}
```

Access: Writing/Display/Print array elements

Similar to the syntax of reading values into array elements, you can display a single value from array using the cout object as follows:

```
cout<<scores[n-1]
```

For example, the following statement may be used to displays the fifth element from the scores array:

```
cout<<scores[4];
```

To display multiple values from array elements, use the cout object and the for loop. For example, the following for loop displays values from the scores array:

```
for(int i=0; i<10;i++){
```

```
    cout<<i+1<<": "<<scores[i]<< endl;
```

Implementation of insert a new element

```
#include<iostream>
using namespace std;
int main(){
    int scores[]={52, 78, 75, 68, 88, 63, 75, 90,78};
    int item=10, k=3, len=9, j=len-1;
    while(j>=k){
        scores[j+1]=scores[j];
        j--;
    }
    scores[k]=item;
    for (int index=0; index<len+1; index++){
        cout<< scores[index]<<endl;
    }
    return 0;
}
```

Memory allocation

Array allocates contiguous memory. Thus if the address of the first element of an array of integers is **X** then the address of the second element will be **X+4** (4 is the size of one integer)) and third will be **X+4+4** and so on. This means that the memories of all elements of an array are allocated together and are continuous.

Question: What is the size of the following array?

```
int marks[9]={12,5,18}
```



Example: one dimensional Array declaration, initialisation and access of its elements

```
#include<iostream>
using namespace std;
int main(){
    /*To initialise directly the array
    the number of values inside {} should not be greater
    than the number of the size of the array*/
    int marks[9]={52, 78, 75, 68, 88, 63, 75, 90,78};
    // To print a given alement. eg. the third element.
    cout<< marks[2]<<endl;
    // Printing elements of the big size array, use the loop
    for(int i=0; i<9; i++)
        cout<<marks[i]<<" ";
    |
    cout<< endl;
    /*When the element at a agiven position is not initialised
    its value at that index become zero*/
    int scores[9]={52, 78, 75}
    for(int j=0; j<9; j++)
        cout<<scores[j]<<" ";
    cout<< endl;
    return 0;
```

```
75
52 78 75 68 88 63 75 90 78
52 78 75 0 0 0 0 0 0
```

Example 2: Declare and initialise array after

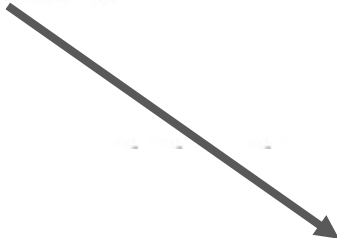
```
1  #include<iostream>
2  using namespace std;
3
4  int main(){
5      |
6      int scores[10];
7      int i,j;
8      /* initializing elements of array scores */
9      for ( i = 0; i<10; i++ )
10         {
11             cout << "Enter value of scores[" << i << "]"<< endl;
12             cin >> scores[i];
13         }
14     /* printing the values of elements of array scores */
15     for ( j = 0; j < 10; j++ )
16     {
17         cout << "scores[" << j << "] = " << scores[j] << endl;
18     }
19     return 0;
20 }
21
```

```
Enter the value of scores[0]
12
Enter the value of scores[1]
14
Enter the value of scores[2]
15
Enter the value of scores[3]
0
Enter the value of scores[4]
11
Enter the value of scores[5]
16
Enter the value of scores[6]
20
Enter the value of scores[7]
18
Enter the value of scores[8]
17
Enter the value of scores[9]
16
scores[0] = 12
scores[1] = 14
scores[2] = 15
scores[3] = 0
scores[4] = 11
scores[5] = 16
scores[6] = 20
scores[7] = 18
scores[8] = 17
scores[9] = 16
```

Example: Array Garbage values

If you access an array which is not initialised or with index beyond the size of the array, there is no warning or error. Instead the program will give you "garbage" data or it will crash.

```
7  int marks[9];  
8  /*Printing elements of the the array not initialised  
9  Garbage values will be printed*/  
10 for(int i=0; i<20;i++)  
11 cout<<marks[i]<<" ";  
12 cout<< endl;
```



```
-2056271376 32604 0 0 -855029152 22066 -855029648 22066 -1039063056 32766 130291  
3792 868381261 -855029152 22066 -2064041065 32604 -112 -1 -1039063048 32766
```



Exercises

- 1a) Write a C++ program to calculate the sum and average of array **scores**. The size of the array should be provided as user input. Array elements also should be provided as user input.
- b) Find the largest and smallest element in the array.
- c) Print the array's elements in reverse(right to left)
- d) Generate a Reversed array of the initial array

3.1.9 Array of characters

To easily handle strings, C++ Standard Library implements string data type that is very useful in handling strings of characters. Because a string is made up of a group of characters, we can also represent them as arrays of char elements using the syntax:

char name[elements];

For example, to declare an array of characters called greetings, use the following syntax:

char Greeting[10];

It is important to note that an array has few characters elements than its size because the last element must store a special character signals end of the string. This special character denoted by '\0' (backslash and zero) is called null

character.



Array of characters(next)

It is important to note that an array has few characters elements than its size because the last element must store a special character signals at the end of the string. This special character denoted by '\0' (backslash and zero) is called null character.

```
#include <iostream>
using namespace std;
int main(){
    char Greeting[30];
    cout << "Greet someone:";
    cin.get(Greeting, 30); //enter 29
    characters
    cout << "Greetings:"<<Greeting<< endl;
    return 0;
}
```

The program uses cin and get() function separated by a period to read characters and store a string of 29 characters. Note that in this case, the 30 th element is reserved for the null terminator \0 that denotes the end of a string.



Array of characters(next)

Here below example of character array:

Example:

```
char str[11]="C++";
```

The array size needs to be designated statically. More memory can't be distributed at run time whenever required.

**1character=1byte
str[11]=>11bytes**

0	1	2	3	4	5	6	7	8	9	10
C	+	+	\0							
The array str has a length of 11										

Null character will terminate the array. If the null character is reached, reading stops there. If str[4]='y', the value of str array will still be C++ because e reached the null character. Then if we change str[3]='x' the char array will print **C++XY** output

Access character array's elements: The loop is not necessary

Initialisation

Similar to the syntax of initialising array of numbers, we can initialise an array of characters with some predetermined sequence of characters within {} braces as follows:

char name[elements]={.,.,.,'\0'}; ex. **char Greeting[6]={`H`,`e`,`l`,`l`,`o`,`\0'};**

Note that, although Hello string has five characters, the sixth element is used to hold `\\0` that signals end of the string. However, instead of initialising an array with comma-separated characters in {} braces, you can declare and initialise a string as follows:

char Greeting[]="Hello";



Reading and writing array of characters

The cin object consists of special function such as get() used to read a valid sequence of null-terminated characters from the input stream. Normally, cout statement and string library functions may be used to display a string, substring or characters.

```
#include <iostream>
using namespace std;
int main(){
char buffer[80];
cout << "Enter a string:";
cin.get(buffer, 79); //enter 79 characters
cout << "String you typed is:"<<buffer<<
endl;
return 0;
}
```

Question: What is the difference between cin, cin.get and cin.getline() when used to record array of characters?



Question 2: What is the output?

```
1. #include <iostream>
2. #include <cstring>
3. using namespace std;
4. int main() {
5.     char String1[] = "Love your neighbour";
6.     char String2[80]= "Promote Peace";
7.     cout<< "String2 before copying: " << String2 << endl;
8.     strcpy(String2,String1);
9.     cout<< "String1 is: " << String1 << endl;
10.    cout<< "String2 after copying: " << String2 <<endl;
11.    cout<< "String2 has: " <<strlen (String2)<< "characters\n";
12.    return 0;
13. }
```

Note: Arrays

Since an array is a collection of homogeneous or same type elements, this means that an array can store all integers, all floating point numbers, all characters (array of character or strings) or any other complex data type, but all of **the same type**. Let's see some important points to be pointed out about arrays.

- 1) An array is always stored in consecutive memory locations.
- 2) An Arrays can store multiple values which can be referenced by a single name, unlike a simple variable which stored one value at a time.
- 3) The array name is actually **a pointer** to the first location of the memory block allocated to the name of the array.
- 4) An array either be an integer, character or floating data type can be initialized only during declaration time and not afterwards.
- 5) There is no bound checking concept for arrays in C++. it means you can attempt to enter any number of values irrespective of the integer index specified inside square brackets, during the declaration of arrays. It is up to programmers to check upper bound or array.

3.1.10. 2D Array

Structure and Algorithms

a) Introduction

A Two dimensional Array is a collection of a fixed number of elements (components) of the same type arranged in two dimensions.

The two dimensional array is also called a matrix or a table. The intersection of a column and a row is called a cell. The numbering of rows and columns starts by 0

Example:

The array `a[3][4]` is an array of 3 rows and 4 columns. In matrix representation, it looks like the following:

	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

`a[i][j]` means that it is an element located at the intersection of row `i` and column

b) Two dimensional array declaration

The syntax for declaring a two-dimensional array is:

SET Array name<-Array [row size][column size] of Data type

Where **row size** and **column size** are expressions of positive integer values, the two expressions row size and column size specify the number of rows and the number of columns, respectively, in the array.

Examples:

1. SET marks<-Array[2][7] of Integer
2. SET scores<-Array[4][3] of Integer

Notice that the number of elements =**row size*column size**

The examples above defines respectively arrays named marks and scores with 14 ($2*7$) elements and 12 ($4*3$) elements. The expression marks [0] [0] will access the first element of the matrix marks and marks [1] [6] will access the last row and last column of array marks.

If the array marks[4][3] is filled with the elements 80, 75, 76, 75, 54, 72, 55, 70, 65, 85, 35 and 59, we get the following table.

	Column 0	Column 1	Column2
Row 0	80	75	76
Row 1	75	54	72
Row 2	55	70	65
Row3	85	35	59

c) Two dimensional array initialization

Two-Dimensional array is initialized by specifying bracketed values for each row. For the case of array of integers, the default values of each element is 0 while for others the default values for other types is not known.

For the case of an array of 3 rows and 4 columns, if the name is A and the type of elements is integer, the initialization is:

```
int A[3][4]={
    {A[0][0], A[0][1], A[0][2], A[0][3]}, /* initializers for row indexed by 0 */
    {A[1][0], A[1][1], A[1][2], A[1][3]}, /* initializers for row indexed by 1 */
    {A[2][0], A[2][1], A[2][2], A[2][3]} /* initializers for row indexed by 2 */
};
```

Note: The nested braces, which indicate the intended row, are **optional** The following initialization is equivalent to the previous initialization.

```
int A[3][4]={A[0][0], A[0][1], A[0][2], A[0][3], A[1][0], A[1][1], A[1][2], A[1][3], A[2][0], A[2][1], A[2][2], A[2][3]};
```

An example of marks for 3 students in 4 courses. The array will look like this:

```
int marks[3][4]={  
    {80, 75, 76, 75},  
    {54, 72, 55, 70},  
    {65, 85, 35, 59}  
};
```

Or

```
int marks[3][4]={80, 75, 76, 75,54, 72, 55, 70,65, 85, 35, 59};
```

	Col0	Col1	Col2	Col3
R0	80	75	76	75
R1	54	72	55	70
R2	65	85	35	59

If the table is called marks, marks [1][3]=70; where the row position is 1 and the column position is 3. The expression marks [0] [0] will access the first element 80 of the matrix marks and marks [2] [3] will access the last row and last column.

Syntax to initializing two-dimensional arrays one by one ^{DSA&C++}

The two
dimensional array
marks which will
hold 12 elements
can be initialized
like:



BEGIN

```
SET marks=Array[3] [4] of Integer  
marks[0][0]=80  
marks[0][1]=75  
marks[0][2]=76  
marks[0][3]=75  
marks[1][0]=54  
marks[1][1]=72  
marks[1][2]=55  
marks[1][3]=70  
marks[2][0]=54  
marks[2][1]=72  
marks[2][2]=55  
marks[2][3]=70
```

END



Q1: 1. Four secondary schools in RUHANGO District are displaying the number of students per year that they registered in 2015, 2016, 2017 and 2018.

- a. How many rows and columns will be in the table that contains the displayed data?
- b. Generate a table that can holds the above mentioned data
- c. Declare an array called population to contain the needed data.
- d. Initialize the array population with numbers of your choice

Q2: Given the array below

A[0][0] 1	A[1][0] 54
A[0][1] 2	A[1][1] 23
A[0][2] 3	A[1][2] 11
A[0][3] 13	A[1][3] 45
A[0][04] 17	A[1][04] 31
A[0][5] 34	A[1][5] 65
A[0][6] 19	A[1][6] 26

- Give the number of elements of the above array;
- Declare array A;
- Initialize the above array

3.4 Accessing two dimensional array element

Activity;

1. *WRITE marks[1][2] ;*
2. *What are the meanings of the above statements*

For example READ marks[1][2] stores a value in second row, third column location of the array marks.

To store multiple values into array you must use a FOR loop.

Syntax:

```
FOR i←0 TO rows-1 DO
    FOR j←0 TO columns-1 DO
        READ array name[i][j]

    END FOR
END FOR
```

Where i refers to the row position and j refers to the column position.

Initial value=lower limit

Example: Write an algorithm of a program that allows the user to enter (store) two dimensional array elements

BEGIN

SET marks=Array [3] [8] of Integer

Use variable i as integer

Use variable j as integer

FOR i←0 TO 2 DO

FOR j←0 TO 7 DO

READ marks[i] [j]

END FOR

END FOR

END

Writing (displaying) two dimensional array elements

Writing (displaying) two dimensional array elements

To access two dimensional array elements' values, we can also take the elements' values of an array residing in the memory of the computer and write them in any output device or a given file.

To write (display) elements from an array in an output device or other file, use a WRITE function or other versions of it depending on the programming languages used for programs writing together with the array name and location (index) of the element. In that case the elements of a two dimensional array can be displayed by the following syntax:

WRITE/PRINT ArrayName [i][j]

Where **i** refers to the row position and **j** refers to the column position.

Algorithm for reading and printing values in arrays:

Example: Write an algorithm that allows the user to enter(store) two dimensional array elements and displays those stored elements.

BEGIN

SET scores=Array [3][8] of Integer

Use i As Integer

Use j As Integer

WRITE “ Enter two dimensional Arrays Elements”

FOR i←0 TO 2 DO

 FOR j←0 TO 7 DO

 READ Score[i][j]

 END FOR

END FOR

Write two dimensional Arrays Elements

FOR i←0 TO 2 DO

 FOR j←0 TO 7 DO

 WRITE Score[i][j]

 END FOR

END FOR

END

Exercise

1. Write an algorithm to do the following operations on a two dimensional array A of size m x n.
 - A. Declare the array A;
 - B. Read the elements (values) of A
 - C. Output the elements (values) of A;
 - D. Calculate the sum of all elements (values) of A;
 - E. OUTPUT the calculated sum
2. Write a programme in C++ Language to implement the algorithm above.

3 Consider a 2 by 3 array T

- a. Write a declaration of T
- b. How many rows does T have?
- c. How many columns does T have?
- d. How many elements does T have?
- e. Write name of elements in second row
- f. Write the name of element in third column
- g. Write a single statement that sets the elements of T in row 1 and column 2 to zero
- h. Write a statement that inputs the values of T from keyboard
- i. Write a statement that displays the elements of first row of T

How 2D is stored in Memory?

Suppose we want to store int marks[5][5].

In memory this 2D array will be stored as a 1D Array

How?

Example: Assume we want to store 7 as marks[2][1]=7

You have to skip=2rows*1 column

In each row, we have 5 elements; the number of cells to skip=2*5+1=11 cells. Then we insert in **12th cell** at **index 11**

If R is rows and C columns, A[i][j] will be inserted at **$i * C + j$** index.

In a 2D array like int A[R][C], The formula for finding index for A[i][j]= $i * C + j$, That is why the value of C is mandatory. To create a 2 Dimensional Array or pass a 2 Dimensional Array to a function, the number of columns is mandatory because our 2 Array is stored in memory as 1D Array and we have to find index using $i * C + j$.

marks[2][1]=7

	7			

0	1									11								24
										7								24



3.1.11 Implementation using C++

How to create, Initialise Print a 2D MATRIX Row Wise

```
#include <iostream>
using namespace std;
int main(){
    int marks[100][100];
    int m, n; //How many Rows , How many Columns
    cin>>m>>n;
    for (int i=0;i<m;i++){ //Iterating over the rows
        for (int j=0;j<n;j++){// Iterating over the columns
            cin>> marks[i][j];
        }
    }
    //Printing row wise
    for (int k=0; k<m; k++){
        for (int t=0; t<n;t++){
            cout<<marks[k][t]<<" ";
        }
        cout<<endl;
    }
}
```

Print Row wise says:
Loop over the rows
and for each row print
each column

Rows
Columns

	Col1	Col2	Col3
R1	1	2	3
R2	4	5	6

How to create, Initialise and Print a 2D MATRIX:Column Wise

```
#include <iostream>
using namespace std;
int main(){
int marks[100][100];
int m, n; //How many Rows , How many Columns
cin>>m>>n;
    for (int i=0;i<m;i++){ //Iterating over the rows
        for (int j=0;j<n;j++){ // Iterating over the columns
            cin>> marks[i][j];
        }
    }

    //Printing column wise wise
    for (int k=0; k<n; k++){
        for (int t=0; t<m;t++){
            cout<<marks[t][k]<<" ";
        }
        cout<<endl;
    }

    return 0;
}
```

Column wise says
Loop over the column and for
each column, print each row

Columns

Rows

	Col1	Col2	Col3
R1	1	2	3
R2	4	5	6



Initialising Two D Arrays

If we have 1D array `int marks[4]`. This can be initialised with `int a [4]={1,3,4,6}`

If we have 2D array `int marks[2][3]`, this can be initialised by

`Int marks[2][3]={{16,12,17},{15,11,19}};`

Giving rows is optionally but giving columns is mandatory

`Int marks[ ][3]={` This is valid

`{13,14,15}.`

`{16,17,13}.`

`{18,14,18}`

`};`

What will happen with the following?

```
Int marks[][3]={{12,14},{11,15,17},{18}};;
```

This is 3*3

12	14	0
11	15	17
18	0	0

Next if `int A[3][4]={{16,15},{12,13}};` 

16	15	0	0
12	13	0	0
0	0	0	0

What is the output?

A

```
#include<iostream>
using namespace std;
void display(int marks[][2], int r, int c)
{
    for (int i=0; i<r;i++){
        for (int j=0;j<c;j++){
            cout<<marks[i][j]<<" ";
        }
        cout<<endl;
    }
}

int main(){

    int A[][2]={
        { 12,14},
        {15,18}
    };
    display(A,2,2);

}
```

B

```
#include<iostream>
using namespace std;
void display(int marks[2][], int r, int c)
{
    for (int i=0; i<r;i++){
        for (int j=0;j<c;j++){
            cout<<marks[i][j]<<" ";
        }
        cout<<endl;
    }
}

int main(){

    int A[][2]={
        { 12,14},
        {15,18}
    };
    display(A,2,2);

}
```

C

```
#include<iostream>
using namespace std;
void display(int marks[][5], int r, int c)
{
    for (int i=0; i<r;i++){
        for (int j=0;j<c;j++){
            cout<<marks[i][j]<<" ";
        }
        cout<<endl;
    }
}

int main(){

    int A[][5]={
        { 12,14},
        {15,18}
    };
    display(A,2,5);
    int B[5][5]={0};
    display(B,5,5)
    return 0;
}
```



Array functions

Array function are the functions that are used to perform operations on set of array. To access array elements, C++ provides various array functions like **at()**, **get()**, **front()**, **back()**, **size()**, **max_size()**, and many more where **at()** will access the array element using array index, **front()** will return first array element, **back()** will return last array element, **size()** will return the number of array elements, **max_size()** is used to show the maximum number of elements.

Array Size in C++

Basically, when we say the length of an array actually we refer to the total number of elements present in the corresponding array. For example, for an array as given below:

```
int array1[] = { 0, 1, 2, 3, 4 };
```

The size of the array or length of the array here is equal to the total number of elements in it. Which, in this case, is '5'.

There are different ways following which we can find the length of an array in C++, they are as follow:

1. Counting element-by-element,
2. `begin()` and `end()`,
3. `sizeof()` function,
4. `size()` function in STL,
5. Pointers.

1. Counting element-by-element

Traversing throughout the given array and simultaneously counting the total number of elements we traversed can give us the length of the array right?

But for traversing we cannot use a for loop directly if we do not know the array length beforehand. This issue can be solved by using a simple for-each loop. Look at the code below carefully.

Note

*1. The use of **auto** keyword. The auto keyword specifies that the type of the variable that is being declared will be automatically deducted from its initializer. In the case of functions, if their return type is auto then that will be evaluated by return type expression at runtime. Good use of auto is to avoid long initializations when creating iterators for containers.*

2: For loop shortened for(auto i: arr) , for(int i: arr)

```
#include<iostream>
using namespace std;
int main()
{
    int c;
    int arr[]={1,2,3,4,5,6,7,8,9,0};
    cout<<"The array is: ";
    for(auto i: arr)
    {
        cout<<i<<" ";
        c++;
    }
    cout<<"\nThe length of the given Array is: "<<c;

    return 0;
}
```

The array is: 1 2 3 4 5 6 7 8 9 0
The length of the given Array is: 10



Using begin() and end()

The length of an array can be calculated using the standard library's `begin()` and `end()` functions. The two functions return iterators pointing to the start and the end of the corresponding array respectively. Take a close look at the given code,

```
#include<iostream>
#include<array>
using namespace std;
int main()
{
    //Given Array
    int arr[] = { 11, 22, 33, 44 };

    cout<<"The Length of the Array is : "<<end(arr)-begin(arr);
    //length

    return 0;
}
```

using sizeof() function to Find Array Length in C++

The `sizeof()` operator in C++ returns the size of the passed variable or data in bytes.

Similarly, it returns the total number of bytes required to store an array too. Hence, if we simply divide the size of the array by the size acquired by each element of the same, we can get the total number of elements present in the array.

```
using namespace std;
int main()
{ //Given array
  int arr[] = {10 ,20 ,30};
  //length calculation

  int al = sizeof(arr)/sizeof(arr[0]);    cout <<
  "The length of the array is: " <<al;

  return 0;
}
```



Array and Pointers

Arrays and pointers

The name of an array is a pointer to the first element in the array.

```
int a[10];  
int *aptr = a;  
aptr = a ;// is equivalent to aptr =  
&a[0];  
aptr+5;// is equivalent to &a[5]  
*(aptr+5); // is equivalent to a[5]
```

```
1  #include <iostream>  
2  using namespace std;  
3  int main()  
4  {  
5      float arr[10];  
6      float *ptr;  
7      cout << "Displaying address using arrays: " << endl;  
8      for (int i = 0; i < 10; ++i)  
9      {  
10         cout << "&arr[" << i << "] = " << &arr[i] << endl;  
11     }  
12     // ptr = &arr[0]  
13     ptr = arr;  
14     cout<<"\n Displaying address using pointers: "<< endl;  
15     for (int i = 0; i < 10; ++i)  
16     { cout << "ptr + " << i << " = "<< ptr + i << endl;  
17     }  
18     getchar();  
19     return 0;  
20 }
```

```
Displaying address using arrays:  
&arr[0] = 0x7ffcdab8dce0  
&arr[1] = 0x7ffcdab8dce4  
&arr[2] = 0x7ffcdab8dce8  
&arr[3] = 0x7ffcdab8dcec  
&arr[4] = 0x7ffcdab8dcf0  
&arr[5] = 0x7ffcdab8dcf4  
&arr[6] = 0x7ffcdab8dcf8  
&arr[7] = 0x7ffcdab8dcfc  
&arr[8] = 0x7ffcdab8dd00  
&arr[9] = 0x7ffcdab8dd04
```

```
Displaying address using pointers:  
ptr + 0 = 0x7ffcdab8dce0  
ptr + 1 = 0x7ffcdab8dce4  
ptr + 2 = 0x7ffcdab8dce8  
ptr + 3 = 0x7ffcdab8dcec  
ptr + 4 = 0x7ffcdab8dcf0  
ptr + 5 = 0x7ffcdab8dcf4  
ptr + 6 = 0x7ffcdab8dcf8  
ptr + 7 = 0x7ffcdab8dcfc  
ptr + 8 = 0x7ffcdab8dd00
```

Array vs Pointer: Program to print the sum of array

```
#include <iostream>
using namespace std;
int main()
{
    int sum,i;
    int a[5]={10,20,30,40,50};
    for (sum = *a, i = 1; i < 5; i++)
        sum+= *(a + i);
    cout<<sum<<endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main()
{
    int sum,i;
    int a[5]={10,20,30,40,50};
    Int *p;
    p=a;
    for (sum = *a, p = a+1; p < a+5; p++)
        sum += *p;

    cout<<sum<<endl;
    return 0;
}
```



Functions and pointers

Question: Write a program with a function that takes the array scores and its size n as the number of elements then RETURN the array which is reversed.

Example:

Consider inputs:

scores[5]={10,20,30,40,50}

The function should return

reversed[5]={50,40,30,20,10}

```
int* reversing4(int scores[],int n){
    int* reversedarray=new int[n];
    int start=0;
    int end=n-1;
    while(start<end){
        reversedarray[start]=scores[end];
        reversedarray[end]=scores[start];
        start++;
        end--;
    }
    if(n%2!=0){
        reversedarray[start]=scores[start];
    }
    return reversedarray;
}
```

```
#include<iostream>
using namespace std;
```

```
int* reversing1(int *arr, int n){
    int *reversed =new int[5];
    for(int i=n-1, j=0;i>=0;i--, j++){
        reversed[j]=arr[i];
    }
    return reversed;
}

int* reversing2(int arr[], int n){
    int *reversed =new int[5];
    for(int i=n-1, j=0;i>=0;i--, j++){
        reversed[j]=arr[i];
    }
    return reversed;
}
```

```
int* reversing(int arr[], int n){
    static int reversed[5];
    for(int i=n-1, j=0;i>=0;i--, j++){
        reversed[j]=arr[i];
    }
    return reversed;
}
```

Static Variables

Static variables in a Function:

When a variable is declared as static, space for **it gets allocated for the lifetime of the program**. Even if the function is called multiple times, space for the static variable is allocated only once and the value of the variable in the previous call gets carried through the next function call.

```
int main(){
    int scores[5]={10,20,30,40,50};

    cout<<endl;
    int *dScore=reversing(scores,5);
    for(int i=0;i<5;i++){
        cout<<dScore[i]<<" ";
    }
    return 0;
}
```



11 Comparison of pointers

Pointers may be compared by using relational operators, such as ==, <, and >.

The following program incrementing the variable pointer so long as the address to which it points is either less than or equal to the address of the last element of the array, which is &var[MAX - 1].

```

1  #include<iostream>
2  using namespace std;
3  int MAX=3;
4  int main () {
5      int var[MAX] = {10, 100, 200};
6      int *ptr;
7      // let us have address of the first element in pointer
8      ptr = var;
9      int i = 0;
10     while ( ptr <= &var[MAX - 1] ) {
11         cout << "Address of var[" << i << "] = ";
12         cout << ptr << endl;
13         cout << "Value of var[" << i << "] = ";
14         cout << *ptr << endl;
15         // point to the previous location
16         ptr++;
17         i++;
18     }
19     getchar();
20     return 0;
21 }

```

```

Address of var[0] = 0x7ffcaed54030
Value of var[0] = 10
Address of var[1] = 0x7ffcaed54034
Value of var[1] = 100
Address of var[2] = 0x7ffcaed54038
Value of var[2] = 200

```

